

BootHop — Complete End-to-End Process Flow

Version 2.0 | Confidential | June 2026

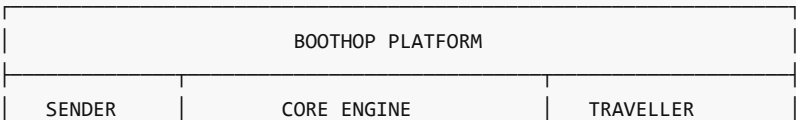
TABLE OF CONTENTS

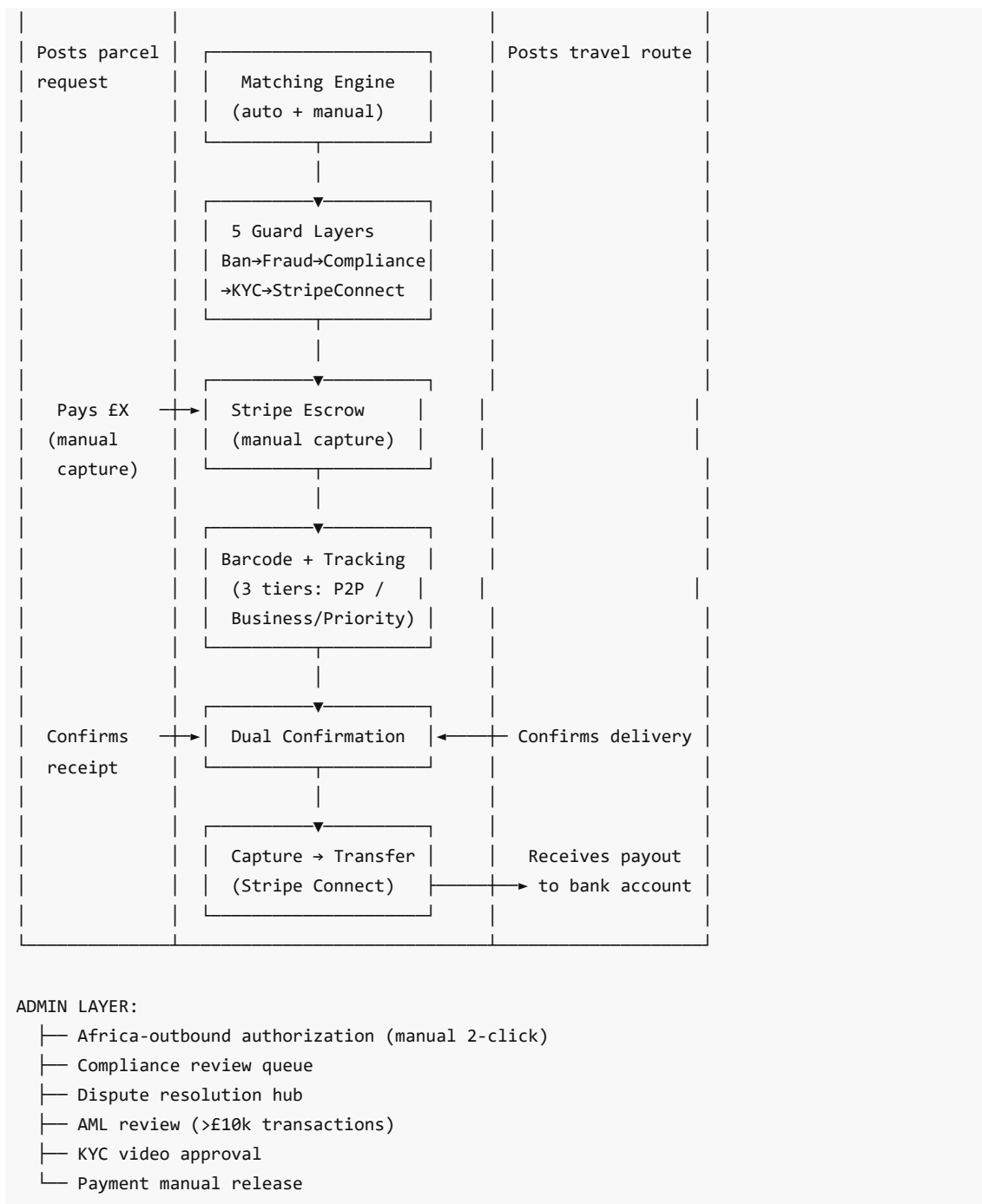
- [1. System Overview & Architecture](#)
- [2. Actors & Roles](#)
- [3. Phase 1 — User Registration & Onboarding](#)
- [4. Phase 2 — Traveller KYC & Stripe Connect](#)
- [5. Phase 3 — Trip Posting](#)
- [6. Phase 4 — Matching Engine](#)
- [7. Phase 5 — Negotiation & Match Response](#)
- [8. Phase 6 — Five Guard Layers \(Fraud, Compliance, KYC\)](#)
- [9. Phase 7 — Payment & Escrow](#)
- [10. Phase 8 — Stripe Webhook Chain](#)
- [11. Phase 9 — Customs & AI Compliance Screening](#)
- [12. Phase 10 — Africa-Outbound Admin Authorization](#)
- [13. Phase 11 — In-Transit Tracking & Barcodes](#)
- [14. Phase 12 — Ghost Detection & Escalation](#)
- [15. Phase 13 — Delivery Confirmation & Payout](#)
- [16. Phase 14 — Disputes & Refunds](#)
- [17. Phase 15 — Ratings & Completion](#)
- [18. Business / Enterprise Tier](#)
- [19. Admin Hub](#)
- [20. Notification System \(Multi-Channel\)](#)
- [21. Fraud & Safety Architecture](#)
- [22. Delivery Tier System \(P2P / Business / Priority\)](#)
- [23. Money Flow & Fee Structure](#)
- [24. Exception Handling & Fallbacks](#)
- [25. Cron Job Schedule](#)
- [26. Full API Route Map \(91 Endpoints\)](#)
- [27. Database Table Reference](#)
- [28. Environment & Integration Map](#)
- [29. Match Status State Machine](#)
- [30. WhatsApp & D818 Pipeline](#)

1. SYSTEM OVERVIEW & ARCHITECTURE

BootHop is a **compliance-first, peer-to-peer logistics network** connecting:

- **Senders (Hoopers)** — people who need a parcel delivered locally or internationally
- **Travellers (Booters)** — people already making a journey who carry parcels for earnings
- **Businesses** — companies needing on-demand, time-critical, or cross-border delivery





2. ACTORS & ROLES

Actor	Also Called	Description	Key Requirement
Sender	Hooper	Posts a parcel delivery request	Valid payment method
Traveller	Booter	Carries parcels on existing journeys	Stripe Connect (KYC verified)

Business	Enterprise	Company using BootHop for logistics	Business account + priority tier
Admin	Ops Team	BootHop operations — approvals, disputes	ADMIN_SECRET access
Stripe	—	Payment processor & escrow holder	Live API keys
Supabase	—	Database + Auth	Service role key
Resend	—	Email delivery	API key
Telnyx	—	SMS, voice calls, OTP	API key + messaging profile
Anthropic (Claude)	—	AI customs item classification	API key
Upstash Redis	—	Webhook idempotency + retry queue	REST URL + token
Meta / WhatsApp	—	WhatsApp notifications + approvals	Phone number ID + access token
Google Maps	—	Address + airport autocomplete	Maps API key

3. PHASE 1 — USER REGISTRATION & ONBOARDING

3.1 Account Creation

```
User visits boothop.com
|
|— Chooses role at registration: Sender or Traveller
|   → /register?type=send OR /register?type=travel
|
|— Signs up via Supabase Auth (email + password)
|   → auth.users record created
|   → public.users profile created
|   → £20 signup credit applied (user.credit_balance = 2000p)
|
|— Verification email sent → /verify
|   → User clicks link to confirm email
|
|— Terms acceptance required
|   → POST /api/terms/accept
|   → users.terms_accepted = true, terms_accepted_at = now()
|
|— Device fingerprint collected silently (every page load)
|   → DeviceFingerprint.tsx component (invisible)
|   → Collects: UA, language, screen, timezone, CPU cores,
|       canvas SHA-256 hash, WebGL vendor/renderer
|   → POSTed to /api/fingerprint
|   → Upserted into device_fingerprints table (email + fingerprint)
```

- Checked: is device banned? shared with banned accounts?
- sessionStorage flag prevents re-firing same session

3.2 OTP / Phone Login (Alternative Auth)

User chooses OTP login:

POST /api/auth/request-code

- Generates time-limited OTP code
- Sent via email (Resend) or SMS (Telnyx)

POST /api/auth/verify-code

- Code validated
- Session created
- Redirected to dashboard

4. PHASE 2 — TRAVELLER KYC & STRIPE CONNECT

Travellers MUST complete this before they can accept matches.

4.1 Stripe Connect Onboarding

Traveller visits /dashboard or prompted after match accept

```
|
|— POST /api/stripe/create-connect-account
|   → stripe.accounts.create({ type: 'express' })
|   → stripe.accountLinks.create({ type: 'account_onboarding' })
|   → Returns Stripe-hosted onboarding URL
|
|— Traveller completes on Stripe:
|   → Government ID upload
|   → Bank account / debit card details
|   → Address verification
|
|— Stripe fires: account.updated webhook → /api/webhooks/stripe-connect
|   → charges_enabled = true
|   → payouts_enabled = true
|
|— UPDATE users:
|   stripe_connect_id          = acct_xxxxx
|   stripe_connect_charges_enabled = true
|   stripe_connect_payouts_enabled = true
|   stripe_onboarding_completed  = true
|   stripe_verification_status   = 'verified'
|   can_receive_payments         = true
|
|— POST /api/stripe/refresh-account-status (polls if needed)
|
|— KYC confirmation email sent to traveller
```

4.2 Video KYC (Enhanced Verification)

For high-value or high-risk matches:

POST /api/kyc/create-session

- Creates Stripe Identity verification session
- Returns session client_secret

User completes video ID on /kyc/[matchId] or /kyc/video

- Stripe Identity live video verification
- Passport / driving licence scan

Stripe fires: identity.verification_session.verified

- /api/webhooks/stripe-identity
- users.stripe_verification_status = 'verified'
- match.kyc_status updated
- KYC approval email sent

Admin review (if flagged):

POST /api/admin/kyc/video-approve

- Manual approval/rejection
- Notifies user

5. PHASE 3 — TRIP POSTING

5.1 Sender Posts a Delivery Request

/requests/create → POST /api/trips/create

Fields submitted:

- from_city, to_city (with Google Maps autocomplete)
- from_country, to_country
- travel_date (when parcel must arrive)
- item_category (electronics, clothing, documents, etc.)
- goods_description (text description)
- declared_value (£ value, used for insurance)
- desired_price (what sender wants to pay)
- weight (kg), dimensions
- special_instructions

Processing:

- AI customs classifier runs (Claude Haiku)
 - classifies item → risk_level (low/medium/high)
 - checks prohibited items list
- INSERT trips (type: 'send', status: 'active')
- auto_created: false
- Trip immediately visible in matching pool

5.2 Traveller Posts a Travel Route

/journeys/create → POST /api/trips/create

Fields submitted:

- from_city, to_city
- from_country, to_country

```
travel_date          (departure date)
available_space (kg)
max_weight
price_wanted          (minimum earnings)
transport_mode        (flight, road, rail)
flight_number (optional)
```

Processing:

- INSERT trips (type: 'travel', status: 'active')
- auto_created: false
- Trip visible in matching pool

Draft trips:

- Can save as draft (status: 'draft')
- GET /api/drafts → lists drafts
- POST /api/trips/publish-draft → makes active

6. PHASE 4 — MATCHING ENGINE

6.1 Auto-Match Cron (runs daily)

```
GET /api/cron/auto-match
```

```
|
|— Query all active 'send' trips
|— Query all active 'travel' trips
|
|— Compatibility scoring for each pair:
|   Route match      (from/to alignment)      → up to 40 pts
|   Date proximity   (within ±2 days)         → up to 30 pts
|   Price proximity   (within 20% of desired) → up to 20 pts
|   Space fit        (weight within capacity) → up to 10 pts
|
|— Score ≥ 70 → CREATE MATCH
|
|— INSERT matches:
|   status:          'matched'
|   sender_trip_id   traveler_trip_id
|   sender_email      traveler_email
|   offered_price     agreed_price
|
|— Creates mirror trip (auto_created: true) for expressing party
|
|— EMAIL both parties:
|   → Sender:      "A traveller wants to carry your parcel"
|   → Traveller:   "Match found for your route"
```

6.2 Manual Interest Expression

```
User clicks "I'm interested" on browse page
POST /api/matches/express-interest
```

- Mirror trip created (auto_created: true)
- Match record created (status: 'matched')
- Notification sent to listing owner

6.3 Follow-Up Unmatched (Cron — 10am daily)

```
GET /api/cron/follow-up-unmatched
→ Finds trips with travel_date = yesterday, still 'active' (no match found)
→ EMAIL trip owner: "Your trip wasn't matched – post again"
→ Deduped per travel_date to prevent repeat emails
```

7. PHASE 5 — NEGOTIATION & MATCH RESPONSE

7.1 Negotiation (Optional)

Either party can negotiate price before accepting:

```
POST /api/matches/send-negotiation-email
→ Sends price counter-offer email to other party
→ Both see proposed price in match details (/matches/[id])
→ Negotiation logged in matches table (counter_price)
```

7.2 Match Response

```
Listing owner opens /matches/[id]
POST /api/matches/[id]/respond
  body: { action: 'accept' | 'decline' }
```

Prerequisites:

- Must be the listing owner (non-auto_created trip side)
- Match must be in 'matched' status

ON DECLINE:

- match status → 'declined'
- Mirror trip deleted
- trips restored to active pool
- Other party notified by email

ON ACCEPT:

- Runs all 5 guard layers (see Phase 6)

8. PHASE 6 — FIVE GUARD LAYERS

All run in sequence on match accept. Any failure stops the process.

GUARD LAYER 1 – BAN CHECKS (parallel database queries)

```
isIpBanned(request_ip)
→ Queries banned_ips table
```

→ Checks expires_at (temporary bans supported)

isAccountBanned(owner_email)

→ Queries banned_accounts table

isAccountBanned(traveller_email)

→ Both parties checked independently

Result: Any ban → 403 "Your access has been restricted"

GUARD LAYER 2 – FRAUD ENGINE (5-factor weighted score)

All 5 sub-scores run in parallel:

identityScore (30% weight)

Stripe onboarding not completed? +30 pts

charges_enabled = false? +20 pts

payouts_enabled = false? +20 pts

verification_status ≠ 'verified'? +10 pts

Max: 100 pts

behaviourScore (30% weight)

Each past cancellation: +10 pts (max 50 pts)

Each open ghost incident: +30 pts (max 60 pts)

Max: 100 pts

transactionScore (20% weight)

agreed_price > £500: +20 pts

agreed_price > £1,000: +20 pts

goods_value > £1,000: +20 pts

goods_value > £3,000: +20 pts

Max: 100 pts

routeScore (20% weight)

Either party in NG/GH/KE/SN/CI/CM/UG/TZ: +60 pts (high risk)

Either party in CN/IN/PK/BD/ET/EG/MA: +30 pts (medium risk)

All other routes: 0 pts

deviceBonus (up to +30 extra points on final score)

Banned fingerprint linked to this account: +40 pts

Same device fingerprint shared across 2+ accounts: +20 pts

No suspicious device signals: 0 pts

WEIGHTED TOTAL:

score = (identity×0.30) + (behaviour×0.30)

+ (transaction×0.20) + (route×0.20)

+ (deviceBonus × 0.75)

Score	Tier	Automated Action
-------	------	------------------

0-29	LOW	Proceed normally
30-59	MEDIUM	Proceed + fraud flag logged
60-79	HIGH	IP banned + proceed + admin alert
80-100	CRITICAL	Match BLOCKED + IP banned + account banned + admin alerted → 403 returned to user

All evaluations logged to fraud_flags table (non-fatal)

GUARD LAYER 3 – ITEM COMPLIANCE

checkItemCompliance(itemCategory, fromCountry, toCountry)

Prohibited items (always blocked, any route):

weapons, explosives, narcotics, counterfeit goods,
live animals, human remains, radioactive materials,
stolen property, child exploitation material

Route-specific rules (examples):

GB → NG: cash=BLOCK, medicine=REVIEW, electronics=ALLOW

GB → GH: same as NG

UK → AE: alcohol=BLOCK, pork=BLOCK, all else=ALLOW

* → *: default risk level applied

AI Classification (Claude Haiku):

→ Classifies item description → category + risk_level

→ HIGH_VALUE_LUXURY: jewellery >£5k, watches >£2k

→ AML_THRESHOLD: cash equivalents >£10k → AML queue

→ UNCLASSIFIED_ITEM: flags for manual review

Result:

allowed=false → 400 "Item not permitted on this route"

action=REVIEW → Flag for manual admin review

action=ALLOW → Continue

GUARD LAYER 4 – STRIPE CONNECT GUARD

Fetches traveller's user record:

can_receive_payments = ?

stripe_connect_id = ?

stripe_onboarding_completed = ?

No stripe_connect_id:

→ 402 "Traveller has not set up payout account"

→ Returns onboarding URL: /traveller/onboarding

Has ID but can_receive_payments = false:
→ 402 "Traveller's account still being verified (24-48h)"
→ Sender instructed to wait or find another traveller

GUARD LAYER 5 – ACCEPT CONFIRMED

All 4 guards passed:
→ match status: 'matched' → 'agreed'
→ Both parties notified by email
→ Sender directed to /checkout/[matchId]
→ 15-minute window: match expires if no payment

9. PHASE 7 — PAYMENT & ESCROW

9.1 Checkout Creation

Sender visits /checkout/[matchId]
POST /api/payment/create-checkout
body: { matchId }

Price Breakdown:

agreed_price	→ carrier earns
+ platform_fee (3-5%)	→ BootHop revenue
+ insurance (7.5% of goods)	→ BootHop reserve
= hooper_pays	→ sender's total charge

Stripe Checkout Session:

```
mode: 'payment'  
payment_intent_data: {  
  capture_method: 'manual' ← ESCROW: authorised, NOT charged  
}  
metadata: {  
  match_id,  
  carrier_payout (pence),  
  currency: 'gbp'  
}  
success_url: /dashboard  
cancel_url: /matches/[id]
```

→ match status: 'payment_pending'
→ Sender enters card on Stripe-hosted page
→ Stripe 3DS authentication (if bank requires)

9.2 Signup Credit Applied

```
If user has credit_balance > 0:
  → Credit deducted from hooper_pays total
  → Reduces Stripe charge amount
  → £20 signup credit automatically applied
```

10. PHASE 8 — STRIPE WEBHOOK CHAIN

10.1 Security & Idempotency

```
POST /api/webhooks/stripe (all Stripe events arrive here)

STEP 1 – AUTHENTICATION
  Standard: stripe.webhooks.constructEvent(body, sig, WEBHOOK_SECRET)
  Bypass:   Header x-internal-bypass: CRON_SECRET (retry cron only)

STEP 2 – REDIS IDEMPOTENCY
  getWebhookEventStatus(event.id)
  → 'processed': skip, return 200
  → 'processing': skip, return 200 (concurrent lock)
  → null:        setWebhookEventStatus('processing', TTL: 300s)
                  → proceed to handle

STEP 3 – HANDLE EVENT
  On success: setWebhookEventStatus('processed', TTL: 7days)
  On failure: setWebhookEventStatus('failed', TTL: 7days)
               pushFailedEvent(event.id) → Redis retry queue
               Admin email alert sent
```

10.2 Event: `checkout.session.completed`

```
Stripe fires when: Sender completes payment form

Extract match_id from session.metadata
UPDATE matches:
  status:          'escrowed'
  payment_intent_id: pi_xxx
  payment_status:  'escrowed'
  escrowed_at:     now()

INSERT transactions:
  match_id, stripe_session_id, stripe_payment_intent_id
  amount, currency, status: 'escrowed'
  sender_email, traveler_email

Release contact details (both parties now see each other):
  → Sender receives traveller's phone + email
  → Traveller receives sender's phone + email

POST /api/matches/unlock-contact (both sides)
```

```
EMAIL → sender:    sendContactReleasedEmail
EMAIL → traveller: sendContactReleasedEmail
SMS (Business/Priority): Telnix SMS notification
```

10.3 Event: charge.captured

Stripe fires when: `paymentIntents.capture()` is called

```
Find match by payment_intent_id
UPDATE matches:
  payment_status:      'released'
  payment_released_at: now()
```

```
Fetch traveller.stripe_connect_id
stripe.transfers.create({
  amount:      carrier_payout (pence),
  currency:    'gbp',
  destination: stripe_connect_id,
  metadata:    { match_id }
})
```

→ Stripe fires: `transfer.created`

10.4 Event: transfer.created

Stripe fires when: Transfer to traveller initiated

```
Find match by transfer.metadata.match_id
UPDATE matches:      status: 'completed', completed_at: now()
UPDATE transactions: status: 'completed', transfer_id, completed_at
```

```
EMAIL → sender:    "Delivery complete – payment processed"
EMAIL → traveller: "Delivery complete – payout on its way"
In-app notifications created for both parties
```

10.5 Event: payment_intent.payment_failed

```
Find match by payment_intent_id
→ match status reverts to 'agreed'
EMAIL → sender:    "Payment failed – please retry"
Admin alerted
```

10.6 Webhook Retry (Every 15 min)

```
GET /api/cron/process-webhook-queue
popFailedEvents(20) from Redis
For each eventId:
  stripe.events.retrieve(eventId) → full event
POST /api/webhooks/stripe
  x-internal-bypass: CRON_SECRET
```

Success → mark 'processed'
Still failing → admin email alert

10.7 Other Stripe Webhooks

/api/webhooks/stripe-connect → Handles: account.updated (KYC changes)
/api/webhooks/stripe-identity → Handles: identity.verification_session.verified

11. PHASE 9 — CUSTOMS & AI COMPLIANCE SCREENING

11.1 AI Item Classification (Claude Haiku)

Triggered: On trip creation + on match accept

POST /api/customs/categories (GET available categories)
POST /api/customs/estimate (estimate duties)
POST /api/compliance/check (full compliance check)

lib/customs/ai-classifier.ts:
→ Calls Anthropic API (claude-haiku model)
→ Prompt: classify item, assess risk, flag prohibited
→ Returns: { category, risk_level, prohibited, reason }

Risk levels returned:
LOW → standard items, proceed
MEDIUM → flag for review, add customs note
HIGH → admin review required
PROHIBITED → block match entirely

AML threshold (Anti-Money Laundering):
→ Declared value > £10,000
→ Match flagged for admin AML review
→ admin_alerts table: alert_type = 'aml_review'
→ Visible in /admin/customs/aml-review

11.2 Customs Duties Estimation

lib/customs/customs-service.ts + rules-engine.ts:

Input: item_category, declared_value, from_country, to_country

Rules applied:
→ Import duty rates by country pair
→ VAT/GST applicable
→ De minimis thresholds (below which no duty applies)
→ Special trade agreements (UK-EU, etc.)

Output: {
 estimated_duty,

```
estimated_vat,  
total_customs_cost,  
de_minimis_exempt,  
notes  
}
```

- Displayed to sender before posting trip
- Saved in trips table for reference

12. PHASE 10 — AFRICA-OUTBOUND ADMIN AUTHORIZATION

For all matches where the destination is an African country (NG, GH, KE, SN, CI, CM, UG, TZ) — an **additional manual admin authorization step** is required.

```
Match reaches 'agreed' (accepted by listing owner)  
|  
├─ System detects destination is Africa-outbound  
|  
├─ INSERT admin_alerts:  
|   alert_type: 'africa_outbound_review'  
|   match_id, sender_email, traveler_email  
|   route: from_city → to_city  
|   item_category, declared_value  
|  
├─ Admin sees match in /admin/hub (matches tab)  
|   Shows: route, items, declared value, both parties' KYC status  
|  
├─ Admin has 2 options:  
|  
|   POST /api/admin/authorise-match  
|   body: { matchId, action: 'approve' }  
|       → match proceeds to checkout  
|       → Sender directed to payment  
|       → Email to both parties: "Match approved – proceed to payment"  
|  
|   POST /api/admin/authorise-match  
|   body: { matchId, action: 'reject', reason }  
|       → match status → 'blocked'  
|       → Email to both: "Match could not be approved – [reason]"  
|       → No charge made  
|  
└─ Admin target: respond within 4 business hours
```

13. PHASE 11 — IN-TRANSIT TRACKING & BARCODES

13.1 Barcode Generation

```
When match reaches 'escrowed':  
POST /api/tracking/generate-barcodes  
→ Generates two unique QR barcodes:
```

- sender_barcode (sender scans to ping traveller)
- traveller_barcode (traveller uses to log checkpoints)
- Stored in matches.sender_barcode, matches.traveller_barcode
- QR code PNG generated (lib/utils/barcode.ts)
- Barcodes emailed to both parties
- Accessible at /track/[barcode] (public tracking page)

13.2 Tracking Tiers

Feature	P2P	Business	Priority
Location pings	3 per window	5 per window	Unlimited
Location prec.	City	Street	Building
Photo proof	Optional	Required	Required
Live GPS	No	No	Yes
Notifications	Email + Push	SMS + Email	SMS + Phone
Support SLA	Standard	24h	2h
Cost to BootHop	£0.01/event	£0.05/event	£0.10/event
Premium add-on	+£2.00	+£2.00	Included

13.3 Tracking Events

Traveller logs checkpoint:

```
POST /api/tracking/share-location
body: { matchId, checkpoint_type, lat, lng, note, photo? }
checkpoint_types:
  'pickup'      → Parcel collected from sender
  'transit'     → En route (airport, station, etc.)
  'location_check' → Periodic location share
  'delivered'   → Parcel handed to recipient
```

Sender pings traveller for location:

```
POST /api/tracking/scan-sender-barcode
→ Rate-limited per tier (3/5/unlimited pings per time window)
→ Creates location_request record
→ Notifies traveller to share location
```

View tracking timeline:

```
GET /api/tracking/get-history?matchId=xxx
→ Returns all checkpoints with timestamps
→ Shown on /track/[barcode] (public page, no auth required)
→ Map view of checkpoint locations
→ Polls every 30 seconds for live updates
```

14. PHASE 12 — GHOST DETECTION & ESCALATION

14.1 Ghost Detector (Cron — 4am daily)

```

GET /api/cron/ghost-detector
|
|— Query: escrowed matches with no tracking activity for 48h+
|   (no checkpoints OR location_requests in last 48h)
|
|— For each ghost match:
|   INSERT ghost_incidents:
|     traveller_email, match_id, status: 'open'
|
|   UPDATE matches:
|     status: 'ghost_flagged'
|     ghost_flagged_at: now()
|
|   INSERT admin_alerts:
|     alert_type: 'ghost_traveller'
|     message: "Traveller silent for 48h on [route]"
|
|   SMS → traveller (Telnyx):
|     "URGENT: BootHop – please update delivery status"
|
|   EMAIL → sender:
|     "Your package status is unknown. We're investigating."
|
|   EMAIL → traveller:
|     "URGENT: Update your delivery status immediately"
|
|— EMAIL → admin: summary of all ghost incidents

```

14.2 Delivery Reminders (Cron — 9am daily)

```

GET /api/cron/delivery-reminders
|
|— Standard reminder (< 72h since escrowed):
|   EMAIL → traveller: "Confirm delivery via your dashboard"
|   EMAIL → sender:   "Confirm receipt via your dashboard"
|   Includes 1-click confirm token link
|
|— Urgent reminder (> 72h):
|   EMAIL → both: "URGENT – confirm to release your payment"
|   Admin review triggered

```

15. PHASE 13 — DELIVERY CONFIRMATION & PAYOUT

15.1 Dual Confirmation

```

POST /api/matches/[id]/confirm-delivery
Authenticated: sender or traveller

Match must be in: 'escrowed', 'active', or 'ghost_flagged'

```

Sender confirms:

- UPDATE matches: `sender_confirmed_delivery = true`
- Notification sent to traveller

Traveller confirms:

- UPDATE matches: `traveller_confirmed_delivery = true`
- Notification sent to sender

BOTH confirmed:

- `stripe.paymentIntents.capture(payment_intent_id)`
 - ← Funds NOW leave sender's card (first time charged)
- match status → `'delivery_confirmed'`
- Stripe fires: `charge.captured` (see Phase 8.3)

15.2 Payout Chain

```
stripe.paymentIntents.capture()
  ↓
charge.captured webhook
  ↓
stripe.transfers.create({ destination: traveller.stripe_connect_id })
  ↓
transfer.created webhook
  ↓
match status: 'completed'
  ↓
transactions table updated (transfer_id, completed_at)
  ↓
EMAIL both parties → "Delivery complete"
  ↓
Rating request sent to both parties
  ↓
Traveller's payout arrives in bank
  (Stripe Connect payout schedule: typically 2 business days)
```

15.3 Auto-Payout Fallback (Cron — 2am daily)

```
GET /api/cron/auto-payout
  → Finds: delivery_confirmed matches not yet payout-released after 24h
  → Manually triggers stripe.paymentIntents.capture()
  → match status → 'completed'
  → Rating emails sent
  (Safety net if webhook missed or race condition occurred)
```

16. PHASE 14 — DISPUTES & REFUNDS

16.1 Raising a Dispute

Either party can raise a dispute within 24h of delivery confirmation:

POST /api/disputes/create

body: { matchId, reason, description, evidence? }

INSERT disputes:

match_id, raised_by, reason, status: 'open'

match status → 'disputed'

EMAIL → admin: "New dispute raised on match [id]"

EMAIL → other party: "A dispute has been raised on your delivery"

Admin can view all disputes at: /admin/hub (disputes tab)

16.2 Dispute Resolution

Admin reviews dispute at /admin/hub

POST /api/admin/disputes/resolve

body: { disputeId, resolution: 'refund_sender' | 'release_traveller' | 'split' }

REFUND_SENDER:

→ stripe.paymentIntents.cancel() (if not captured yet)

OR stripe.refunds.create() (if already captured)

→ match status → 'refunded'

→ sendRefundNotifications() → sender email

→ Traveller notified: no payment

RELEASE_TRAVELLER:

→ stripe.paymentIntents.capture() proceeds

→ Transfer made to traveller

→ match status → 'completed'

→ Sender notified: dispute resolved in traveller's favour

SPLIT:

→ Partial capture + partial refund

→ Custom amounts defined by admin

17. PHASE 15 — RATINGS & COMPLETION

17.1 Rating Submission

Both parties invited to rate after completion:

POST /api/matches/[id]/rate

body: { rating: 1-5, comment, role: 'sender'|'traveller' }

INSERT ratings:

match_id, rater_email, ratee_email

rating (1-5), comment, role

Ratings visible on user profiles

- Feeds into future fraud engine behaviourScore
- Displayed on browse pages (traveller star rating)

18. BUSINESS / ENTERPRISE TIER

18.1 Business Authentication

Separate auth system (OTP-based, no password):

POST /api/business/auth/send-otp → SMS/email OTP
POST /api/business/auth/verify-otp → verify + session
GET /api/business/auth/me → business profile
POST /api/business/auth/logout

Business portal: /business/portal

Priority section: /business/portal/priority

18.2 Business Job Flow

Business creates delivery job:

POST /api/business/create-job
Fields: pickup_address, delivery_address
item_description, weight
priority_level (standard/priority/critical)
route_type (UK-UK, UK-EU, EU-UK, UK-Intl)
delivery_mode (airport-airport, door-door, airport-door, door-airport)

Business pricing tiers:

Standard: base rate + weight uplift
Priority: +40% uplift on base
Critical: +100% uplift (AOG parts, pharmaceutical, legal docs)
Insurance: optional add-on

GET /api/business/get-jobs → Available jobs in pool
GET /api/business/my-jobs → Business's own jobs
PATCH /api/business/update-job → Update job details
POST /api/business/cancel-job → Cancel job

Payment:

POST /api/business/checkout → Stripe checkout (same escrow flow)

Status updates:

POST /api/business/update-status → Carrier logs progress

Priority Partner Program:

POST /api/business/priority-apply → Apply for priority tier
POST /api/business/upload-document → Upload compliance docs
/business/priority-partner → Programme overview
/business/priority-partner/payment → Priority membership payment

Business webhook:

POST /api/business/webhook → Receives delivery status updates

19. ADMIN HUB

All admin routes require ADMIN_SECRET header.

19.1 Admin Hub Overview

/admin/hub – Central operations dashboard

Tabs:

- └─ Matches: All pending/flagged matches
- └─ Disputes: Open disputes awaiting resolution
- └─ Payments: Manual payment controls

19.2 Admin Capabilities

Match Authorization:

- GET /api/admin/hub/matches → All pending matches
- POST /api/admin/authorise-match → Approve/reject Africa-outbound

Dispute Resolution:

- GET /api/admin/hub/disputes → All open disputes
- POST /api/admin/disputes/resolve → Resolve with outcome

Compliance Queue:

- GET /api/admin/compliance → Items flagged for review
- POST /api/admin/compliance/approve → Approve flagged item

AML Review:

- GET /api/admin/customs/aml-review → High-value transactions (>£10k)

Payment Controls:

- POST /api/admin/confirm-payment → Manually confirm payment
- POST /api/admin/release-payment → Manually release escrow

KYC Approvals:

- POST /api/admin/kyc/video-approve → Approve/reject video KYC

Business Management:

- /admin/business → Manage business accounts
- /admin/customs → Customs management

20. NOTIFICATION SYSTEM (MULTI-CHANNEL)

20.1 Channel Matrix

lib/services/notifications.ts – Central notification dispatcher

Event	Email	SMS	Voice	Push
Match found	✓	–	–	✓
Match accepted	✓	–	–	✓
Payment escrowed	✓	✓*	–	✓
Ghost detected	✓	✓	–	✓
Delivery reminder	✓	✓*	–	✓
Delivery confirmed	✓	✓*	–	✓
Payout transferred	✓	✓*	–	✓
Dispute raised	✓	–	–	✓
Critical admin alert	✓	✓	✓†	✓
KYC approved	✓	–	–	–
Payment failed	✓	–	–	✓

* Business and Priority tier only

† Priority tier only

20.2 Email Templates (Resend)

All emails sent from: BootHop <noreply@boothop.com>

lib/email/ folder contains:

- sendVerificationEmail.ts → Account email verification
- sendMatchEmail.ts → Match found / accepted / declined
- sendKycEmail.ts → KYC instructions + video link
- sendPaymentEmail.ts → Payment escrowed + contact released
- sendDeliveryEmail.ts → Reminders, checkpoints, completion
- sendRatingEmail.ts → Post-delivery rating request
- sendTermsEmail.ts → Terms acceptance confirmation
- sendBusinessEmail.ts → Business tier notifications
- sendDisputeEmail.ts → Dispute raised / resolved
- sendFollowUpEmail.ts → Unmatched trip follow-up

20.3 SMS (Telnyx)

lib/services/telnyx.ts

sendSMS(to, message)

- From: +447822001981
- Alpha sender: "BootHop"
- International outbound enabled
- Messaging profile: 40019e92-68dd-4108-92b5-7084f386327a

makeCall(to, message)

- Voice call for Priority tier urgent alerts
- Uses TELNYX_CONNECTION_ID

```
sendVerificationCode(phone)
  → OTP via Telnyx Verify
  → Profile: TELNYX_VERIFY_PROFILE_ID

verifyCode(phone, code)
  → Validates OTP
  → updates users.phone_verified = true

Inbound webhook: /api/telnyx/webhooks
  → Handles delivery receipts
  → SMS reply handling
```

20.4 Web Push Notifications

```
lib/services/notifications.ts
  → VAPID-based web push
  → Stored subscription endpoints per user
  → Sent on: match events, delivery updates, payment events
  → Works when user has PWA installed (manifest.json)
```

21. FRAUD & SAFETY ARCHITECTURE

21.1 Eight-Layer Safety Framework

Layer 1: Device Fingerprinting	(silent, on every page load)
Layer 2: IP Ban Check	(on match accept)
Layer 3: Account Ban Check	(on match accept – both parties)
Layer 4: Fraud Score (5-factor)	(on match accept)
Layer 5: Item Compliance Engine	(on trip create + match accept)
Layer 6: AI Customs Classification	(on trip create)
Layer 7: Stripe 3DS Authentication	(on payment)
Layer 8: Dual Delivery Confirmation	(before funds released)

21.2 Risk & Decision Engines

```
lib/riskEngine.ts
  → Scores 0-100 based on: item, country, user history, value, quantity
  → Feeds into overall match risk

lib/decisionEngine.ts
  → Returns: ALLOWED / RESTRICTED / PROHIBITED
  → Considers: riskEngine score + route rules + admin overrides

lib/classifier.ts
  → Categorizes items for risk scoring
  → Maps free-text descriptions → standard categories
```

22. DELIVERY TIER SYSTEM

Tier determined at match creation based on:

- └─ premium_tracking add-on purchased (£2.00)?
- └─ Business account?
- └─ Priority partner status?
- └─ Delivery count (frequent users upgrade)
- └─ Declared value (high value → higher tier)
- └─ Pickup type (airport = premium)

lib/services/tier-manager.ts:

P2P TIER (standard consumer):

location_requests: 3 per time window
precision: city-level
photo_required: false
notifications: email + web push
tracking_cost: £0.01 per event
support: standard (48h)

BUSINESS TIER:

location_requests: 5 per time window
precision: street-level
photo_required: true
notifications: SMS + email + push
tracking_cost: £0.05 per event
support: 24h SLA

PRIORITY TIER:

location_requests: unlimited
precision: building-level
photo_required: true
live_gps: true
notifications: SMS + phone call + email + push
tracking_cost: £0.10 per event
account_manager: dedicated
support: 2h SLA

PREMIUM TRACKING ADD-ON (any tier):

- +£2.00 one-time
- Unlocks premium features for that delivery

23. MONEY FLOW & FEE STRUCTURE

23.1 Fee Breakdown

Sender pays:

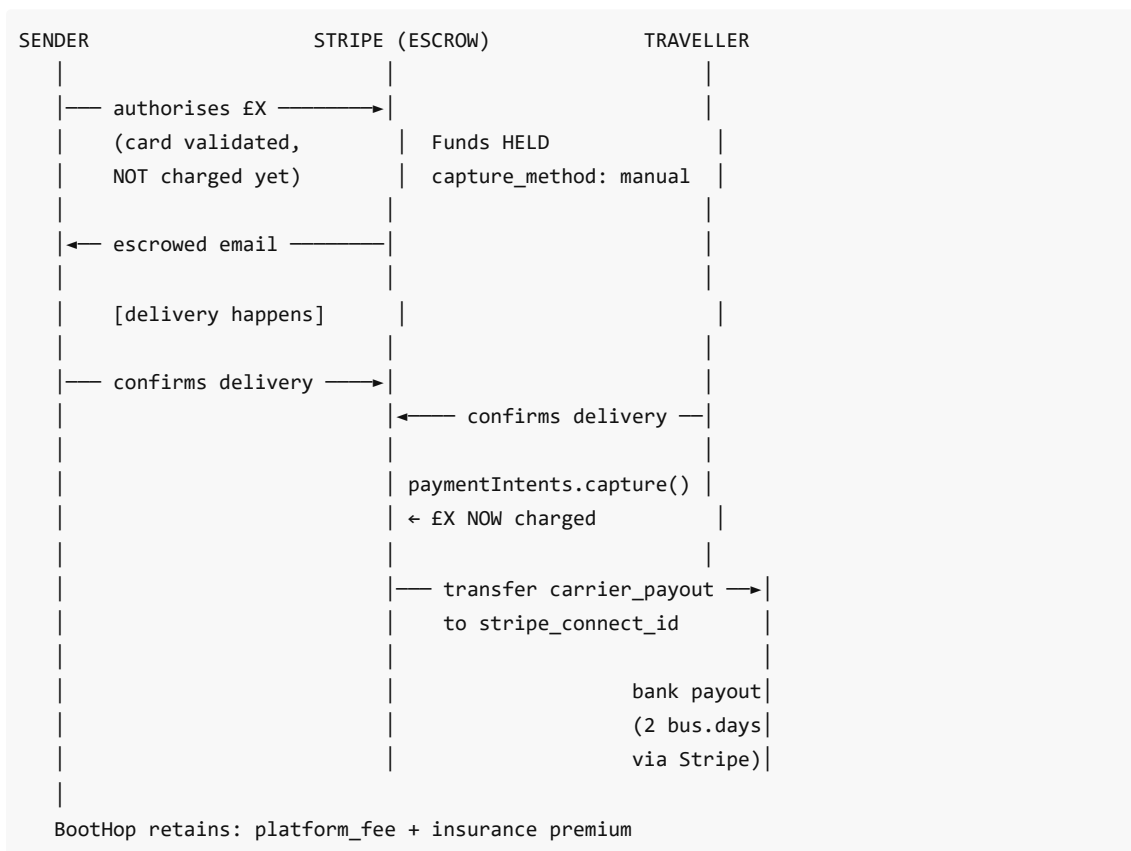
agreed_price	→ base delivery fee	
+ platform_fee_sender	→ 3% (Hooper side)	
+ platform_fee_traveller	→ 5% (Booter side, built-in)	
+ insurance premium	→ 7.5% of declared goods value	

- signup_credit	→ £20 first-time discount
= hooper_pays	→ TOTAL charged to sender's card

Traveller receives:

agreed_price	→ what was agreed
- platform_fee_traveller	→ 5% BootHop cut
= carrier_payout	→ transferred to Stripe Connect

23.2 Complete Money Flow



24. EXCEPTION HANDLING & FALLBACKS

Scenario	How Detected	Automated Action	Admin Action
Card declined	payment_intent.payment_failed webhook	Sender emailed to retry; match stays 'agreed' 24h	None needed
Match not accepted (4h)	expire-matches cron	Match expired, trips restored	None needed

Payment not made after accept (12h)	expire-matches cron	Match expired, trips restored	None needed
KYC pending too long (72h)	expire-matches cron	Match expired	None needed
Traveller goes silent (ghost)	ghost-detector (48h)	Ghost incident, SMS + email, admin alert	Review in admin hub
Stripe webhook fails	Redis retry queue	Retried every 15 min, admin alerted	Check admin alerts
Duplicate webhook event	Redis idempotency	Skipped silently	None needed
Critical fraud score (80+)	Fraud engine on accept	Match blocked, IP + account banned	Receives admin_alert
AML threshold exceeded (>£10k)	AI classifier	Match flagged for AML review	Review in /admin/customs
Africa-outbound not approved	Admin authorization step	Match paused at 'agreed'	Approve/reject in admin hub
Dispute raised	User action	Match paused, both notified, admin alerted	Resolve in /admin/hub
Refund required	Admin resolve	Stripe refund issued	None after resolution
Payout not released (24h)	auto-payout cron	Forced capture triggered	None needed

25. CRON JOB SCHEDULE

Job	Schedule	Purpose
auto-payout	2:00 AM daily	Release held payments after 24h (safety net)
expire-matches	3:00 AM daily	Cancel stale unaccepted matches
ghost-detector	4:00 AM daily	Flag silent travellers, create incidents
delivery-reminders	9:00 AM daily	Chase pending delivery confirmations
follow-up-unmatched	10:00 AM daily	Re-engage senders with no match
process-webhook-queue	Every 15 min	Retry failed Stripe webhook events

26. FULL API ROUTE MAP (91 Endpoints)

Authentication

Route	Method	Description
/api/auth/request-code	POST	Request OTP code
/api/auth/verify-code	POST	Verify OTP + create session
/api/auth/me	GET	Current user info
/api/auth/logout	POST	End session
/api/auth/confirm-action	POST	Email action confirmations
/api/auth/user-trips	GET	User's trips

Trips

Route	Method	Description
/api/trips/create	POST	Create send or travel trip
/api/trips/publish-draft	POST	Publish a draft trip
/api/trips/delete	POST	Delete a trip
/api/drafts	GET	Get draft trips

Matches

Route	Method	Description
/api/match-engine	POST	Run matching algorithm
/api/matches/[id]/details	GET	Match details
/api/matches/[id]/respond	POST	Accept / decline match
/api/matches/[id]/rate	POST	Rate match post-delivery
/api/matches/[id]/confirm-delivery	POST	Confirm delivery
/api/matches/express-interest	POST	Express interest
/api/matches/cancel	POST	Cancel match
/api/matches/send-negotiation-email	POST	Counter-offer email
/api/matches/unlock-contact	POST	Unlock contact details

Messaging

Route	Method	Description
/api/messages/send	POST	Send in-app message
/api/messages/list	GET	List messages for match

Payment & Escrow

Route	Method	Description
/api/create-payment-intent	POST	Stripe payment intent
/api/create-checkout	POST	Stripe checkout session (legacy)
/api/confirm-payment	POST	Confirm payment received
/api/payment/request	POST	Request payment
/api/payment/create-checkout	POST	Create payment checkout
/api/release-payment	POST	Release escrow payment

Stripe Connect

Route	Method	Description
/api/stripe/create-connect-account	POST	Start Stripe Connect onboarding
/api/stripe/add-payment-method	POST	Add payment method
/api/stripe/refresh-account-status	POST	Poll Connect status

KYC

Route	Method	Description
/api/kyc/create-session	POST	Start Stripe Identity session
/api/kyc/video-submit	POST	Submit video KYC

Tracking & Barcodes

Route	Method	Description
/api/tracking/generate-barcodes	POST	Generate QR barcodes
/api/tracking/get-history	GET	Get tracking timeline
/api/tracking/scan-sender-barcode	POST	Sender pings traveller
/api/tracking/share-location	POST	Traveller logs checkpoint

Disputes

Route	Method	Description
/api/disputes/create	POST	Raise a dispute

Customs & Compliance

Route	Method	Description
-------	--------	-------------

/api/customs/categories	GET	List categories
/api/customs/estimate	POST	Estimate customs duties
/api/compliance/check	POST	Run compliance check

Notifications

Route	Method	Description
/api/notifications/send	POST	Send notification

Business Tier

Route	Method	Description
/api/business/auth/send-otp	POST	Business login OTP
/api/business/auth/verify-otp	POST	Verify business OTP
/api/business/auth/me	GET	Business user info
/api/business/auth/logout	POST	Business logout
/api/business/create-job	POST	Create delivery job
/api/business/get-jobs	GET	Available jobs
/api/business/my-jobs	GET	Own jobs
/api/business/update-job	PATCH	Update job
/api/business/cancel-job	POST	Cancel job
/api/business/checkout	POST	Job payment
/api/business/update-status	POST	Update job status
/api/business/priority-apply	POST	Apply for priority tier
/api/business/upload-document	POST	Upload compliance docs
/api/business/webhook	POST	Business webhook receiver

Admin

Route	Method	Description
/api/admin/authorise-match	POST	Authorize Africa-outbound match
/api/admin/hub/matches	GET	All pending matches
/api/admin/hub/disputes	GET	All disputes
/api/admin/compliance/approve	POST	Approve compliance
/api/admin/compliance	GET	Compliance queue

/api/admin/customs/aml-review	GET	AML review queue
/api/admin/confirm-payment	POST	Manual payment confirm
/api/admin/release-payment	POST	Manual escrow release
/api/admin/kyc/video-approve	POST	Approve KYC video
/api/admin/disputes/resolve	POST	Resolve dispute

Webhooks

Route	Method	Description
/api/webhooks/stripe	POST	All Stripe payment events
/api/webhooks/stripe-connect	POST	Stripe Connect events
/api/webhooks/stripe-identity	POST	Stripe Identity (KYC) events
/api/telnyx/webhooks	POST	SMS/call delivery receipts
/api/webhook/whatsapp	GET/POST	Meta WhatsApp Cloud API

WhatsApp & Utility

Route	Method	Description
/api/whatsapp	GET	Open WhatsApp contact link
/api/dashboard	GET	User dashboard data
/api/user/credit	GET	Signup credit balance
/api/airports/search	POST	Airport search
/api/location/check	POST	Validate location
/api/terms/accept	POST	Accept terms
/api/send-verification-email	POST	Resend verification email
/api/send-match-email	POST	Send match email
/api/contact	POST	Contact form
/api/contact/verify	POST	Verify contact form
/api/translate	POST	Translation (Anthropic)
/api/fingerprint	POST	Device fingerprinting
/api/booter-apply	POST	Apply to become a Booter

Cron Jobs

Route	Schedule	Description
-------	----------	-------------

/api/cron/auto-match	Daily	Auto-match algorithm
/api/cron/auto-payout	2am daily	Auto-release escrow
/api/cron/expire-matches	3am daily	Expire stale matches
/api/cron/ghost-detector	4am daily	Detect inactive users
/api/cron/delivery-reminders	9am daily	Delivery reminders
/api/cron/follow-up-unmatched	10am daily	Follow-up unmatched
/api/cron/process-webhook-queue	Every 15m	Retry failed webhooks

27. DATABASE TABLE REFERENCE

Table	Purpose
auth.users	Supabase auth records
users	User profiles, Stripe Connect details, account status, credit
trips	All send + travel trip postings (type: send/travel)
matches	Match records linking sender + traveller trips
transactions	Payment audit ledger (every Stripe event)
device_fingerprints	Browser fingerprints → fraud detection
fraud_flags	Every fraud evaluation with score + factors
banned_ips	IP addresses banned by fraud engine
banned_accounts	Email accounts banned by fraud engine
ghost_incidents	Ghost traveller incidents
admin_alerts	All admin alerts (fraud, ghost, AML, compliance)
disputes	Dispute records
ratings	Post-delivery ratings (1–5 stars)
tracking_checkpoints	Traveller location logs
location_requests	Sender pings for traveller location
notifications	In-app notification records
whatsapp_messages	WhatsApp approval messages (D818 pipeline)
delivery_costs	Tracking cost ledger per tier

Match Status Reference

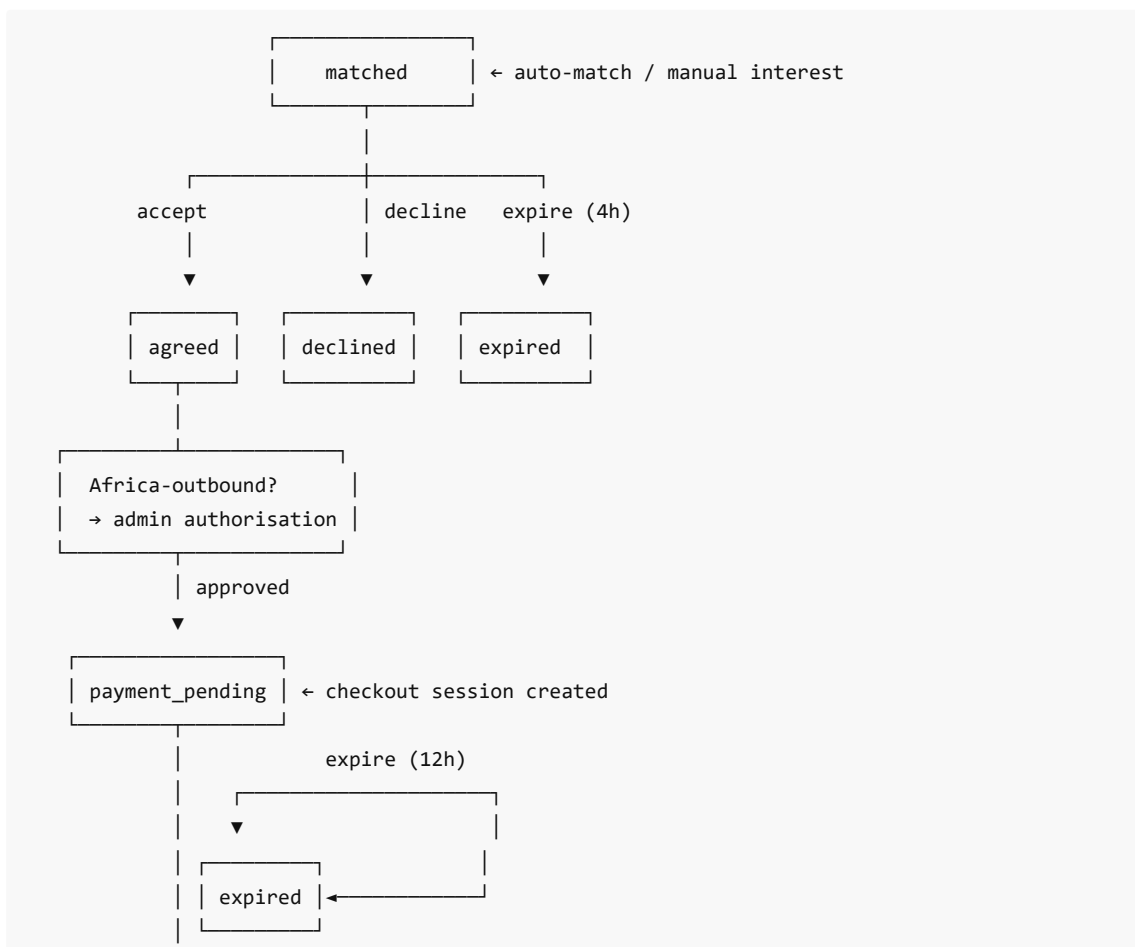
Status	Meaning
matched	Auto-matched or interest expressed, awaiting owner response
agreed	Owner accepted, awaiting payment (or Africa-outbound admin approval)
payment_pending	Checkout session created, awaiting card payment
escrowed	Funds authorised & held, contacts released, in transit
ghost_flagged	No traveller activity for 48h — ghost incident created
delivery_confirmed	Both confirmed, Stripe capture triggered
completed	Transfer made to traveller — delivery done
disputed	Active dispute raised by either party
cancelled	Cancelled by either party
expired	Timed out (no response, no payment, or KYC not done)
blocked	Blocked by fraud engine (critical score)
refunded	Refund issued to sender
declined	Listing owner declined

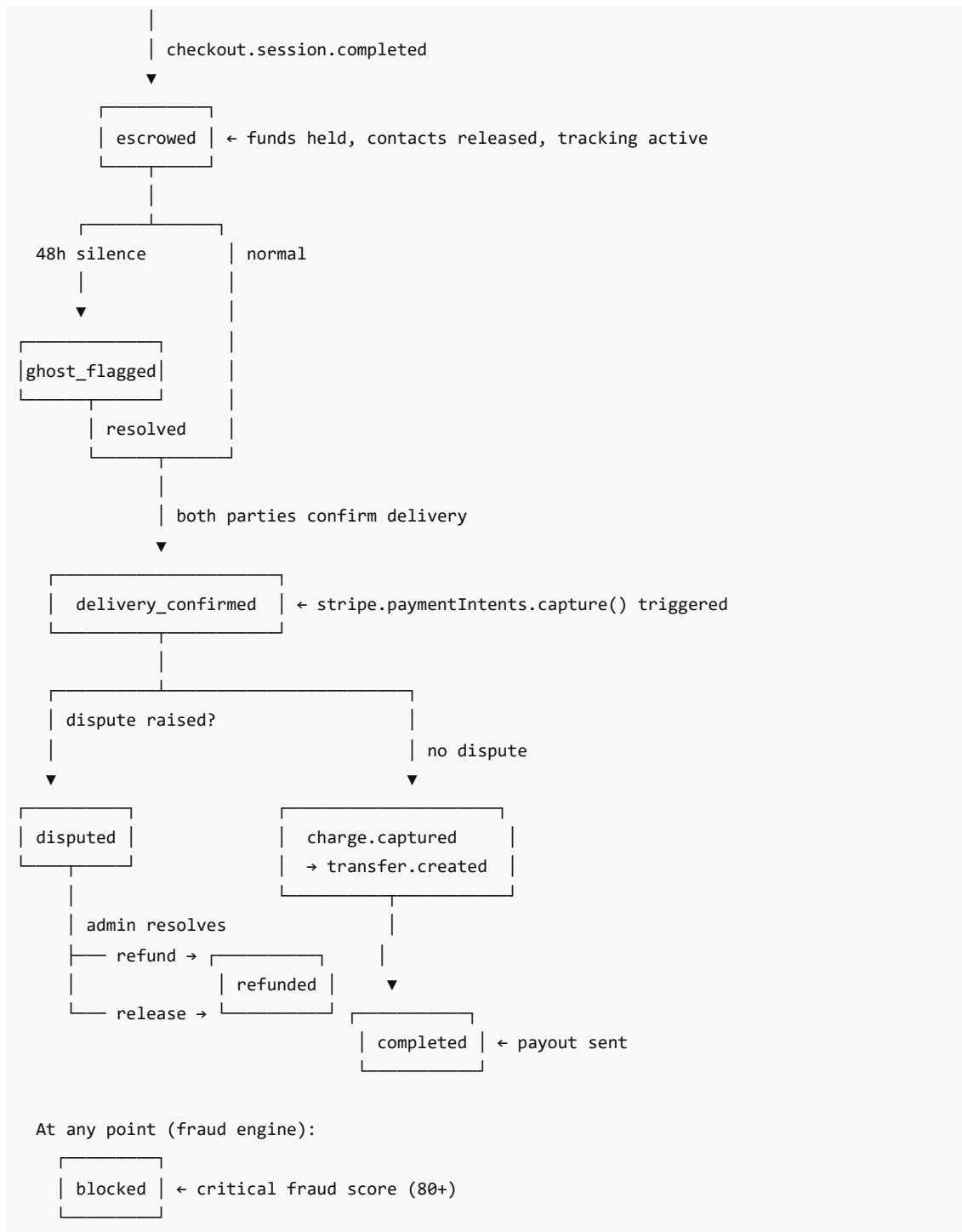
28. ENVIRONMENT & INTEGRATION MAP

Service	Environment Variable	Purpose
Supabase	NEXT_PUBLIC_SUPABASE_URL	Database URL
Supabase	NEXT_PUBLIC_SUPABASE_ANON_KEY	Client queries
Supabase	SUPABASE_SERVICE_ROLE_KEY	Server admin operations
Stripe	STRIPE_SECRET_KEY	Server Stripe API (live)
Stripe	NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Client Stripe.js (live)
Stripe	STRIPE_WEBHOOK_SECRET	Webhook signature verification
Resend	RESEND_API_KEY	All transactional emails
Telnyx	TELNYX_API_KEY	SMS + voice
Telnyx	TELNYX_PHONE_NUMBER	+447822001981 outbound
Telnyx	TELNYX_MESSAGING_PROFILE_ID	SMS messaging profile
Telnyx	TELNYX_CONNECTION_ID	Voice connection
Telnyx	TELNYX_VERIFY_PROFILE_ID	OTP verification
Upstash Redis	UPSTASH_REDIS_REST_URL	Webhook idempotency

Upstash Redis	UPSTASH_REDIS_REST_TOKEN	Redis auth
Anthropic	ANTHROPIC_API_KEY	AI customs classification
Google Maps	NEXT_PUBLIC_GOOGLE_MAPS_API_KEY	Address + airport autocomplete
Google Analytics	NEXT_PUBLIC_GA_ID	Usage analytics
WhatsApp	WHATSAPP_PHONE_NUMBER_ID	Meta Cloud API phone
WhatsApp	WHATSAPP_BUSINESS_ID	Meta Business account
WhatsApp	WHATSAPP_ACCESS_TOKEN	Meta API access
WhatsApp	META_SYSTEM_USER_TOKEN	System user token
Admin	ADMIN_SECRET	Admin API authentication
Admin	ADMIN_EMAIL	Admin alert recipient
App	NEXT_PUBLIC_APP_URL	https://www.boothop.com
Cron	CRON_SECRET	Internal bypass header

29. MATCH STATUS STATE MACHINE





30. WHATSAPP & D818 PIPELINE

30.1 WhatsApp Integration

Used for: Admin approval notifications + D818 internal pipeline

Inbound/Outbound: Meta WhatsApp Cloud API

Phone number: WHATSAPP_PHONE_NUMBER_ID

Business: WHATSAPP_BUSINESS_ID

Webhook handler: /api/webhook/whatsapp (GET + POST)

GET ?poll=1&id=APPROVAL_ID

- Polls for admin responses to approval requests
- Returns: { status: 'pending' | 'approved' | 'rejected' }

POST (inbound messages from Meta):

- Parses incoming WhatsApp messages
- Stores in whatsapp_messages table
- Recognises keywords:
 - "POST" or "YES" → action approved
 - "SKIP" or "NO" → action rejected
- Updates approval status in database

Client redirect:

- GET /api/whatsapp → redirects to whatsapp://send?phone=WHATSAPP_RECIPIENT
- Used for contact links (click to WhatsApp support)

30.2 D818 Pipeline

Internal BootHop operations workflow:

- Africa-outbound approvals can be sent via WhatsApp
- Admin receives WhatsApp message with match details
- Replies POST/SKIP to approve/reject
- System polls /api/webhook/whatsapp?poll=1 to get response
- Match authorised or rejected automatically based on reply